

Household Size Prediction from Amazon Purchases

Annabelle Lee, Jessica Ye, Joshua Rusit, Andrew Chang, Shaina Paryani

I. Introduction

Understanding household composition through consumer behavior has large benefits for corporations like Amazon that operate on a global scale. It provides them insight into their market demographics and allows them to produce targeted advertising strategies. With this Kaggle competition, we are aiming to predict household size with Amazon purchase date and survey responses. Intuitively speaking, with some business acumen, it is likely that larger households will have more total spending, higher order frequencies, and a larger range of diverse products. In our approach, we used machine learning models like Random Forest and XGBoost with hyperparameter tuning to develop a regression model that estimates household size from the purchasing behaviors provided.

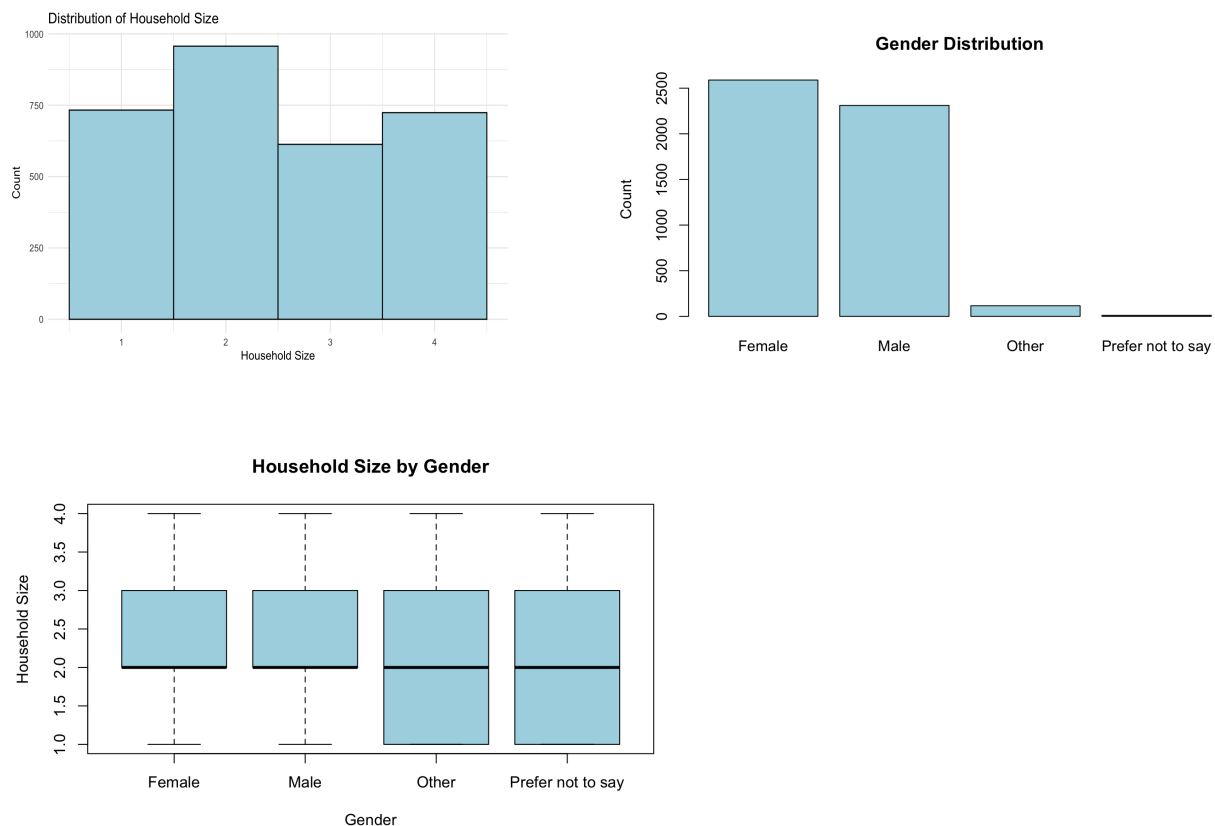
II. Exploratory Data Analysis

We conducted exploratory data analysis on the raw datasets to learn more about the distributions, patterns, and trends before we begin model testing. Our analysis includes 10 visualizations, which we have organized into 3 groups for organized understanding.

Target Variable and Characteristics of the Sample

To begin with, the distribution of household sizes, we can see that 2-person households are the most common, followed by single-person and 4-person households. It is important to note that since 3-person households have the fewest observations, we can expect some inconsistency in distribution means. The gender distribution showed that we have around the same number of

female and male respondents, with a few respondents identifying as “Other” or “Prefer not to say”. We are happy with this result because we now know our models can identify patterns across different demographic groups, like gender. Lastly, the household size by gender boxplot shows that median household sizes are almost the same among the different gender categories. While it may seem like gender alone will not be a useful predictor, we are motivated to find ways to make it valuable through interactions with other features.

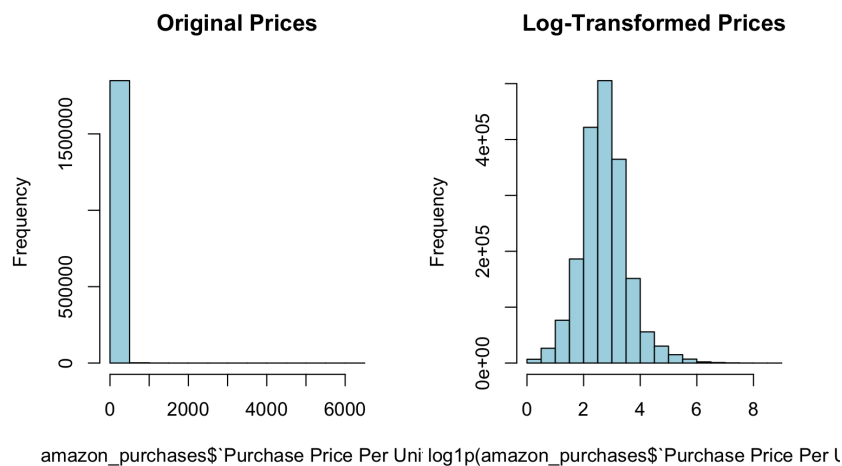


Visualizations of Household Size and Gender Distribution

Looking at purchase patterns

The distribution of purchase prices, which we limited to 100 dollars (otherwise, the visualization was not proving very useful), is right-skewed. Most of the transactions are less than

20 dollars, and the peak is at 10-15 dollars. This informs us that we might have to use log transformation when we preprocess the data. Comparing the original price distribution with its log transformed version, we can see that the log transformation normalizes the right-skewed data with a more symmetric shape.

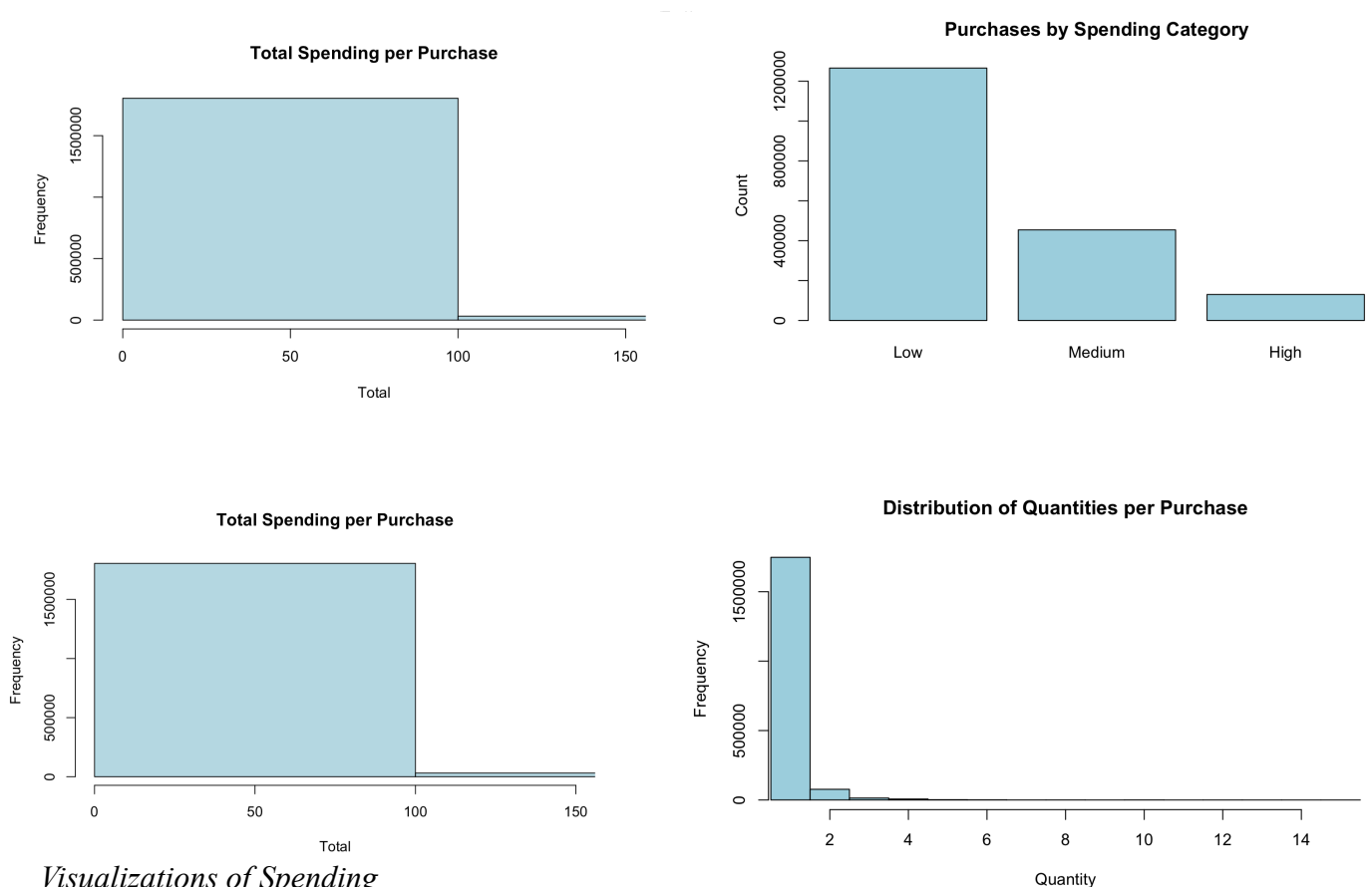


Original Prices VS Log-Transformed Prices

Furthermore, the top 10 product categories graph demonstrates how the most popular Amazon categories are “ABIS_BOOK”, “PET_FOOD”, and “GIFT_CARD”. The categorization is not quite unified because objects that should technically be in the same category aren’t. Thus, we will need to perform some standardization prior to fitting our model(s).

The total spending per purchase visualization, which was again capped at 150 dollars, showed us that most of the transactions were less than 50 dollars. This distribution is left-skewed. Regardless, we are intrigued by this result because we are now further inclined to aggregate spending at the customer level instead of the transactional level. There is not much to deduce from these individual, segmented transactions. Changing continuous spending into categorical groups (low, medium, high) confirms that most spending does indeed fall within the low category.

Finally, the distribution of quantities per purchase graph revealed that most of the purchases made by customers were orders for only 1 item. However, we can see there are still purchases of 5 or more items, and this will prompt us to create somewhat of a ‘large/bulk’ purchase indicator in our feature engineering phase in the hopes that there will be some correlation between the quantity of purchase and household size since that is quite intuitive.



Visualizations of Spending

Shopping Behavior of Customers

The number of purchases per customer were also right-skewed. Once again, we are further inclined to be more specific in our analysis. Instead of using totals and/or sums, we must use frequency rates calculated from this raw data to better inform our analysis. Supporting the creation of temporal patterns is the purchase frequency over time visualization. There was a

dramatic increase in purchase frequency in 2020, which was likely due to COVID and the closure of in-person stores. We cannot group all the data from 2017-2022 as one because it would be inaccurate and misleading. Our last visualization, price vs quantity, demonstrates an inverse relationship i.e. the lower the price, the higher the quantity. Relating back to our discussion on large/bulk purchases, we can now maybe start to see a pattern that lower priced goods and hence large frequency orders are purchased by households with different needs than those who demonstrate the opposite. Examining how purchase quantity varies across spending categories further confirms this interaction pattern. As shown by the quantity by spending category visualization, there are non-linear relationships between spending level and household size.



Visualizations of # of Purchases, Frequency, Price vs. Quantity, and Quantity by Category

III. Preprocessing

In our analysis, we used 2 main datasets, which were the survey dataset with demographic information & household size responses and an Amazon purchase history dataset.

Data Standardization and Feature Creation

We began by creating a custom function to standardize the inconsistent Amazon product categories into eight groups: grocery, baby, kids, pet, book, electronics, apparel, and other using keyword pattern matching on category and title. We created features (parsed date objects, total spending per line, family indicators, low-cost purchases, etc.) at a transactional level so that we could aggregate all individual purchases to a customer level. For each customer, we calculated useful volume metrics, price distribution statistics, category spendings, temporal patterns, and purchase behavior. We needed this information to further explore connections between transactions and customers.

Feature Engineering

After creating the aggregated data, we applied log transformations to right-skewed variables to reduce the impact of outliers. We also created proportional features for each category that tell us about spending composition. As a result, we are able to compare purchases across different income levels. Without this standardization, our results would not prove useful.

Interaction Terms

Along with converting some category survey variables to factors and replacing NA values with column medians, the last key step we took was to create interaction terms. These are

useful in capturing non-additive relationships between predictors. Some examples include purchase diversity x total purchases, total spending x percentage of spending on baby items, and gender x spending level. These numeric and factor based interactions will help the model perform better.

Feature Selection

Our process initially created 52 predictors from the raw Amazon purchase data. However, to reduce model complexity and only select the most predictive features, we trained an initial XGBoost model on the full feature set and computed feature importance scores. After selecting the most significant features and removing any redundant variables, we ended up with 21 features. These set of features put high importance on the interaction terms, the proportional spending by category, temporal shopping pattern, and information about uniqueness of product & purchases. This process is included in the Rmd file for reference but has been commented out.

IV. Candidate models, Model Evaluation and Tuning

The models attempted in our analysis were: Ridge, Lasso, Random Forest, and XGBoost.

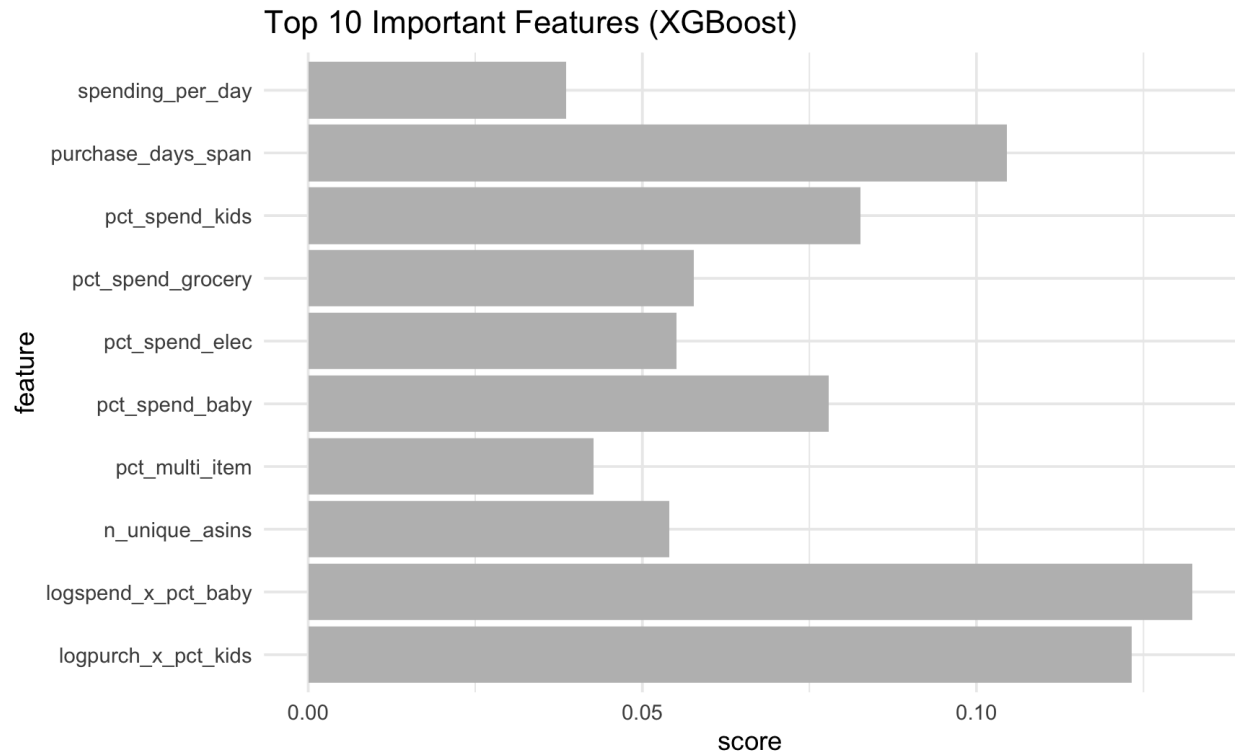
Models	Variables / Features Used	Hyperparameters	Tuning
XGBoost	All features: Q.demos.gender, Q.demos.state, logpurch_x_pct_kids, logspend_x_pct_baby, purchase_days_span, spending_per_day, pct_spend_baby, pct_spend_kids, pct_spend_elec, pct_spend_grocery, pct_spend_pet,	nrounds(150-1800), max_depth(3-9), eta(0.008-0.1), subsample(0.65-0.95), colsample_bytree(0.65-0.95), min_child_weight(2-15), gamma(0-0.8)	random_search (batch_size=25, term_evals=180, 3-fold CV)

	family_spending, pct_multi_item, pct_top_category, purchase_diversity, median_price, pct_pack_orders, n_household_maintenance, repeat_asin_rate, n_unique_asins, log_total_spending		
Lasso	All features	alpha=1, lambda=10 ^{seq(-4,2,0.05)}	CV via cv_glmnet
Ridge	All features	alpha=0, lambda=10 ^{seq(-4,2,0.05)}	CV via cv_glmnet
Random Forest	All features	num.trees(800-1500), mtry.ratio(0.2-0.8), sample.fraction(0.6-1.0), min.node.size(1-10)	random_search (batch_size=20, term_evals=120, 3-fold CV)

Table of the four Candidate Models

XGBoost

XGBoost is another tree-based method, similar to Random Forest, but it builds the trees one after another instead of all at once. Each new tree focuses on the mistakes the earlier trees made by learning from the residuals. It also has built in regularization and tuning steps to keep the model from overfitting. Our model achieved a cross-validated RMSE of 1.042. Therefore, on average, our household size predictions are off by 1.042 people. Two most important features are logpurch_x_pct_kids and logspend_x_pct_baby. This model works the best when Amazon customers have spending patterns that point to them having a family. It overpredicts non-family households.



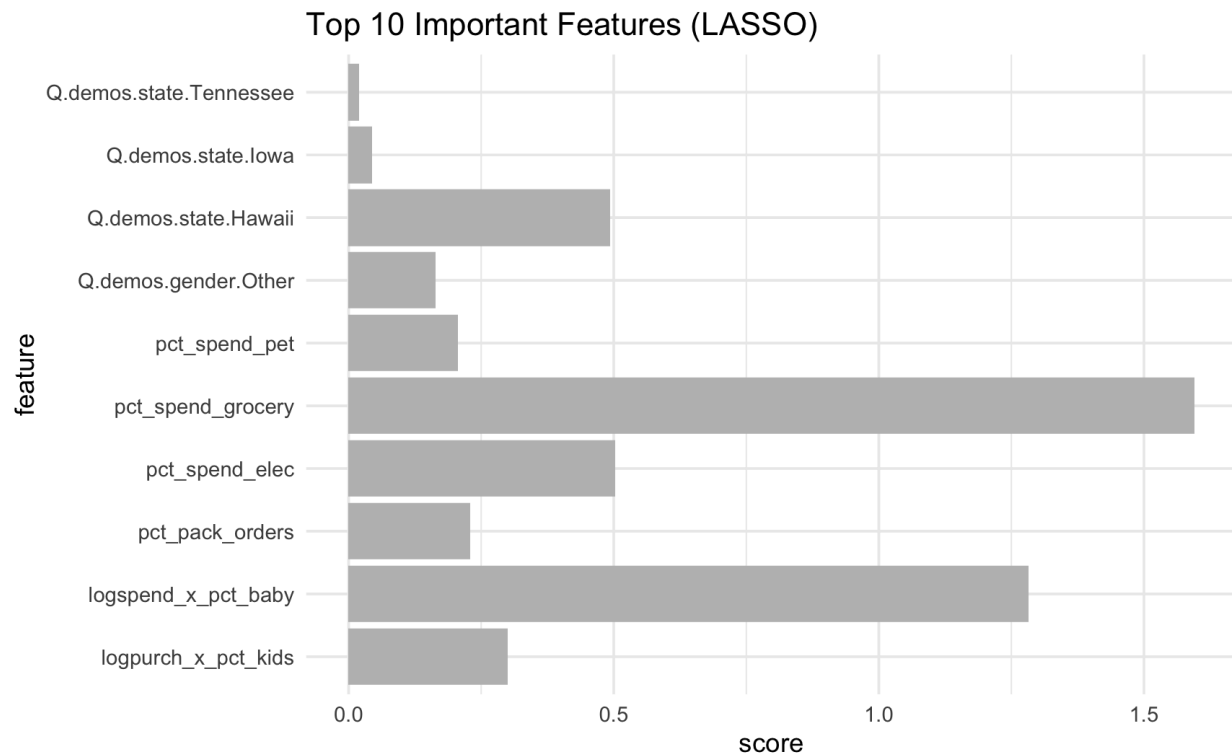
Customer <chr>	Actual <dbl>	Predicted <dbl>	baby_spending <dbl>	kids_spending <dbl>	family_spending <dbl>
Customer 1	1	2.19	0.00	0.00	0.02
Customer 2	3	3.37	0.02	0.24	0.27
Customer 3	1	2.60	0.00	0.03	0.07

XGBoost: Individual Predictions

Lasso

Lasso adds a L1 penalty term $\alpha = 1$ to linear regression. Although the Lasso also shrinks coefficients like Ridge, some coefficients are forced to 0, and this is essentially feature selection. Lasso is useful when we are under the impression that only a small group of a dataset's features will be relevant predictors. The Lasso model achieved a cross-validated RMSE of 1.070, which is a little worse than XGBoost (1.042). It also identified very different features as the most

important, like `pct_spend_grocery`, which is the proportion of spending on groceries. We can conclude that linear models like Lasso may rely more on direct category spending patterns.



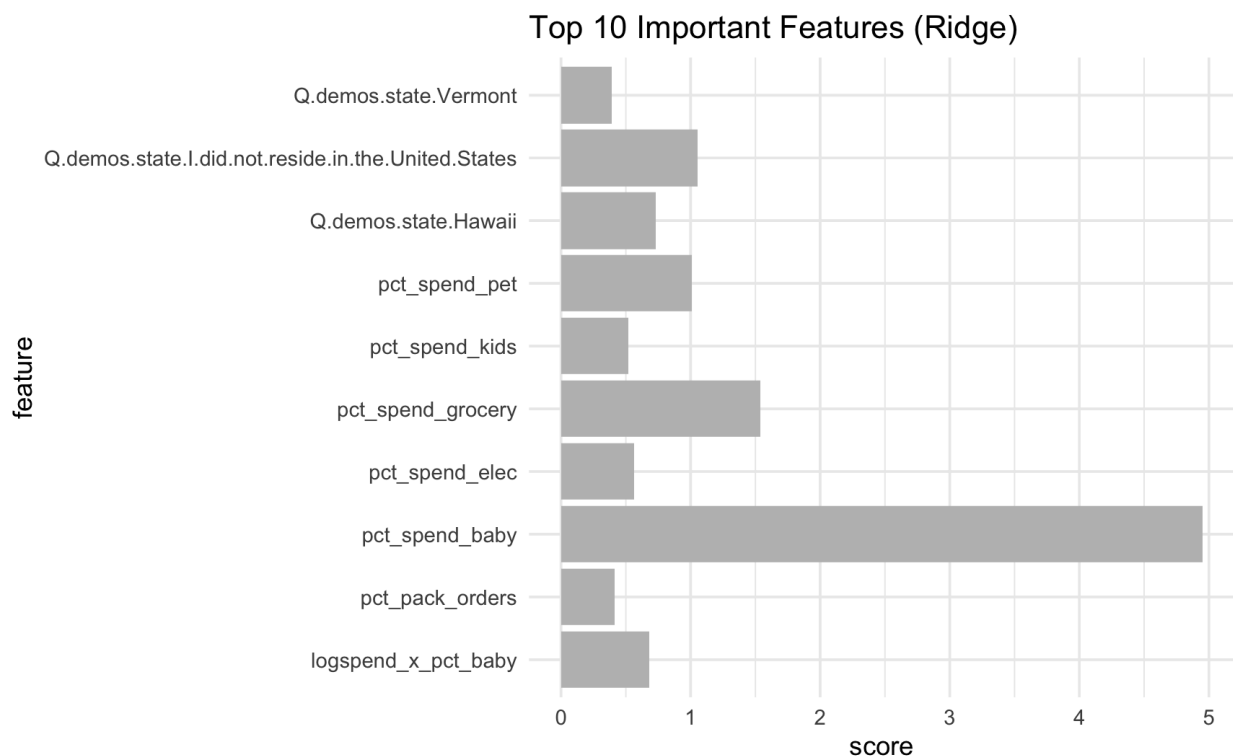
Customer <chr>	Actual <dbl>	Predicted <dbl>	baby_spending <dbl>	kids_spending <dbl>	family_spending <dbl>
Customer 1	1	2.33	0.00	0.00	0.02
Customer 2	3	2.96	0.02	0.24	0.27
Customer 3	1	2.46	0.00	0.03	0.07

Lasso: Individual Predictions

Ridge Regression

This is a standardized linear model that adds a L2 penalty term $\alpha = 0$ to the standard least squares objective function. The penalty term shrinks coefficient estimates to 0, which in turn

reduces model variance and prevents overfitting in the case(s) where we have many predictors. The Ridge model achieved a cross-validated RMSE of 1.078. The feature `pct_spend_baby` dominates the top 10 important features by a large margin, thus indicating that it is the strongest linear predictor for household size. This model would work well with customers that have consistent patterns in their spending habits. Otherwise, not so much, because Ridge doesn't do as well to capture more 'random' patterns.

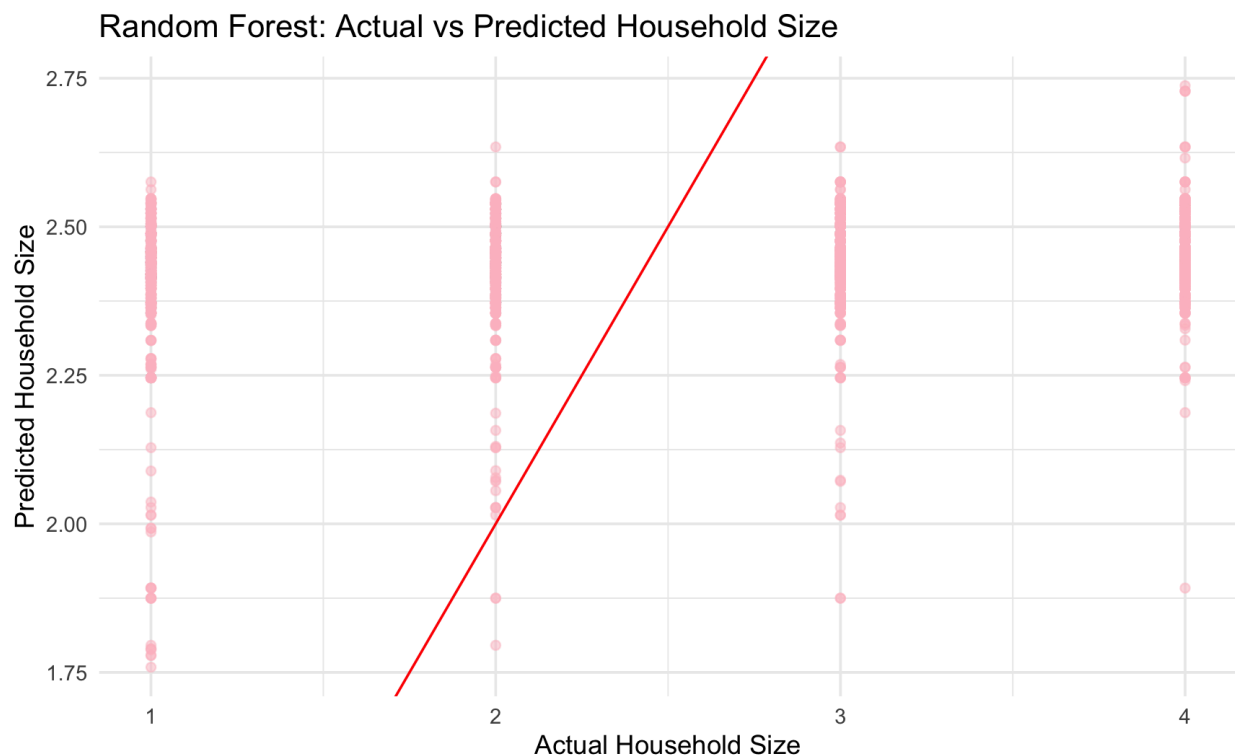


Customer <chr>	Actual <dbl>	Predicted <dbl>	baby_spending <dbl>	kids_spending <dbl>	family_spending <dbl>
Customer 1	1	2.34	0.00	0.00	0.02
Customer 2	3	2.82	0.02	0.24	0.27
Customer 3	1	2.43	0.00	0.03	0.07

Ridge: Individual Predictions

Random Forest

Random Forest is an ensemble method that builds many decision trees on bootstrapped samples of the data. Each split on the tree only considers a random subset of features. The final prediction averages across all these trees, in turn reducing variance and keeping a low bias. Random forest does well to handle non-linear relationships and is good for dealing with outliers/missing values. Since feature importance is challenging for Random Forest due to the mlr3 structure, we visualized prediction quality through an Actual VS. Predicted scatter plot instead. The model achieved a cross-validated RMSE of 1.102; there is strong regression toward the mean and there seems to be some compression as well.



Customer <chr>	Actual <dbl>	Predicted <dbl>	baby_spending <dbl>	kids_spending <dbl>	family_spending <dbl>
Customer 1	1	2.44	0.00	0.00	0.02
Customer 2	3	2.44	0.02	0.24	0.27
Customer 3	1	2.43	0.00	0.03	0.07

Random Forest: Individual Predictions

To conclude, we evaluated 4 different candidate models using 5-fold cross validation. In other words, we split the training data into 5 equal parts and trained each model on 4 folds while validating on the last remaining fold. The last step was to average performance across the folds. For Ridge and Lasso regression, we utilized cross validated lambda selection from the `cv.glmnet` function. For the Random Forest model, we tuned hyperparameters using a random search with 120 different settings (each evaluated with 3-fold CV). We opted for the `tune()` function with random search instead of the `auto_turner()` learnt in class because it gave us more control over the hyperparameters (Rohart et al., 2017). However, since this took a lengthy time to knit, we decided to hardcode the hyperparameters from the best parameters found by our code.

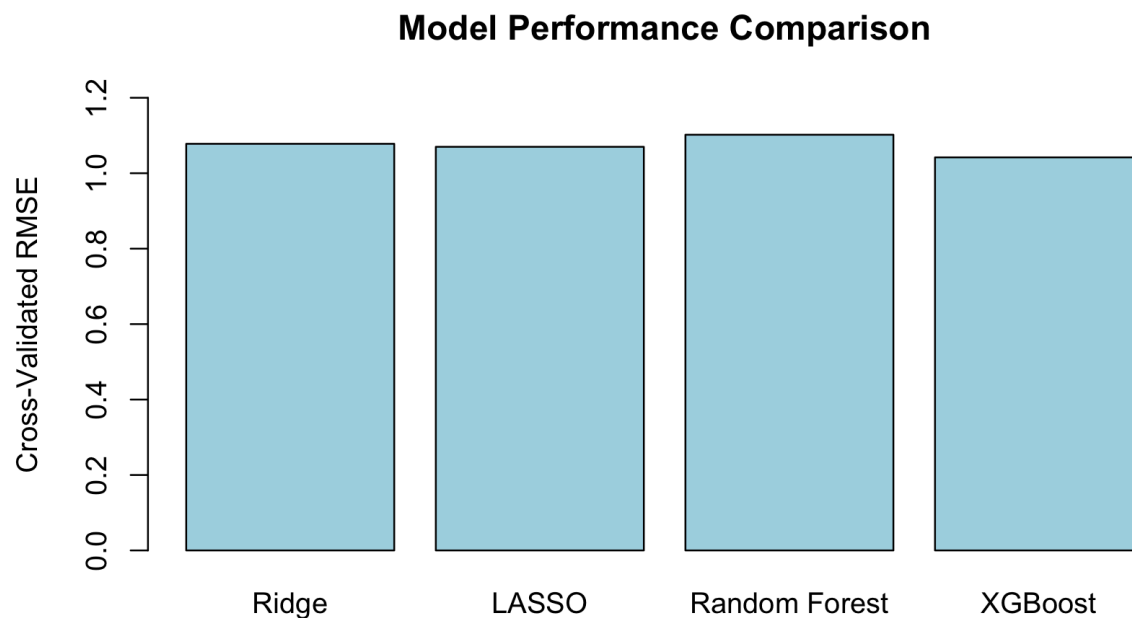
We focused on the following parameters: number of trees, mtry ratio, sample fraction, and minimum node size. This approach worked well since we were able to test out different combinations without the manual additional effort. XGBoost needed the most tuning because its parameters, in comparison to the other candidates, are larger. We ran a random search with 180 trails and adjusted the following parameters: number of boosting rounds, tree depth, learning rate, subsample & column sample ratios, minimum child weight, and gamma. We were hence able to control the complexity of the trees and the learning rate.

Model	RMSE
XGBoost	1.042676
Lasso	1.069658
Ridge	1.077927
Random Forest	1.101465

RMSE of Candidate Models

XGBoost was our best performing model, with a cross-validated RMSE of 1.042. This was 5% better than our ‘worst’ model, Random Forest with RMSE of 1.102. Lasso and Ridge performed almost the same with scores of 1.070 and 1.078 respectively. We can thus conclude that both $\alpha = 0$ and $\alpha = 1$ give benefits, but Lasso is slightly better / more suitable in this scenario. What we found surprising was Random Forest’s poor performance since it is known to be good with non-linear models in comparison to our other candidates. The scatter plot that compared actuals with predicted answered our confusion, since we see that there is a lot of regression towards the mean.

Ni



Visualization of Model Performance Comparison

V. Discussion of final model

We selected XGBoost as our final model based on its lowest RMSE. This model usually has a tendency to capture non-linear relationships well, and feature interaction made it better for this analysis. Intuitively, we know that household size depends on more than just linear regression/association. A key strength of this model was how it automatically learns feature interactions. It was able to identify `logpurch_x_pct_kids` and `logspend_x_pct_baby` as the most important features. Those features combined purchase volume with spendings from each category, and this is a strong predictor as opposed to individual features. It is not difficult to believe either because we assume that those who shop frequently and buy items related to babies/children will have a larger household size because it indicates that they are a family.

Furthermore, XGBoost was able to make good use of the continuous variables like temporal patterns and spending amounts, along with proportional & categorical variables. Its ability to handle mixed data types without having us perform extensive preprocessing is a major advantage (“XGBoost for R introduction”, n.d.). XGBoost gives us feature importance scores, and we can use these scores to truly understand which purchasing behaviors exhibited most strongly predict household size.

Although this model provided us with accurate and useful insight, the individual predictions (although they were only for 3 customers) demonstrate that it overpredicted household size for single person households. This systematic bias tells us the model found it challenging to differentiate between parents who purchase items for their children regularly and ‘normal’ individuals who occasionally buy something from that category. Additionally, there are very few predictions for household sizes below 1.5 or above 4, confirming how it favors predictions near the dataset. This, however, is a common issue in many models other than XGBoost. In terms of computational cost, this model required the most hyperparameter tuning

among all the models and had the longest training time. Despite this not being a major drawback in our scenario, it might be for others or when working with even larger datasets on limited capacity. Along the same lines, a big reason this model performed as well as it did is because of all the preprocessing we handled before even touching the learner code.

There are several ways we could improve the model moving forward. For example, rather than relying on the overall date span, we could analyze purchase dates at the monthly or weekly level. Families would likely show strong purchasing behavior towards children products during the holiday season, back-to-school, etc. to further deconstruct bulk purchasing behavior. We can feature engineers for product size as well. We could also explore adding ZIP code or other geographic variables, which might strengthen the model. However, this isn't certain because it adds another layer of complexity to our raw data. As an additional enhancement, we could pair XGBoost with LASSO to model both linear and non-linear relationships. Another important step would be developing a strategy for dealing with irregular shopping patterns, which tend to distort the analysis.

VI. Appendix

Rohart, F., Gautier, B., Singh, A., & Lê Cao, K.-A. (2017). *mixOmics: An R package for 'omics feature selection and multiple data integration* (R package version 6.3.2). Retrieved from <https://www.rdocumentation.org/packages/mixOmics/versions/6.3.2/topics/tune>

XGBoost for R introduction. (n.d.). *XGBoost documentation*.

https://xgboost.readthedocs.io/en/stable/R-package/xgboost_introduction.html